

1단계 실습 보고서

SocketCAN, VCAN 테스트베드 구축

작성일	2026-08-21
문서 유형	일반 문서
문서 상태	초안

목차

1	가상머신 생성	3
1.1	가상머신 설치 환경 (Host PC)	3
1.2	UTM: Ubuntu 가상머신 생성	3
1.2.1	새로운 가상머신 생성	3
1.2.2	ARM64 ISO 선택과 아키텍처 오류 방지	6
1.2.3	저장소와 가상머신 이름 설정	6
1.3	Ubuntu 설치	12
1.3.1	설치 프로그램 기본 설정	12
1.3.2	네트워크와 미러 설정	12
1.3.3	저장소와 사용자 계정 설정	16
1.3.4	설치 완료와 첫 로그인	18
1.4	SSH 연결	22
2	Docker 없이 vcan0과 can-utils 실행	22
2.1	vcan0 생성과 상태 확인	22
2.2	can-utils 설치	23
2.3	Ubuntu 호스트에서 송수신 확인	23
3	Docker 설치	24
3.1	Docker apt 저장소 설정	24
3.2	Docker Engine과 Compose 설치	25
3.3	Docker 설치 확인과 사용자 권한 설정	25
3.4	Ubuntu 24.04 이미지와 컨테이너 실행	26
4	Docker에서 vcan0 확인	27
4.1	Docker 안에서 cansend와 candump 테스트	28
5	실습 결과	28

1 가상머신 생성

본 실습에서는 Ubuntu Server 가상머신에 SocketCAN 기반의 가상 CAN 인터페이스 vcan0을 구성한다. 먼저 Ubuntu 호스트에서 CAN 프레임 송수신을 확인하고, 이후 Docker 컨테이너에서도 동일한 인터페이스를 사용할 수 있는지 검증한다.

1.1 가상머신 설치 환경 (Host PC)

- Windows: VirtualBox + Ubuntu 24.04 Server
- macOS: UTM + Ubuntu 24.04 Server

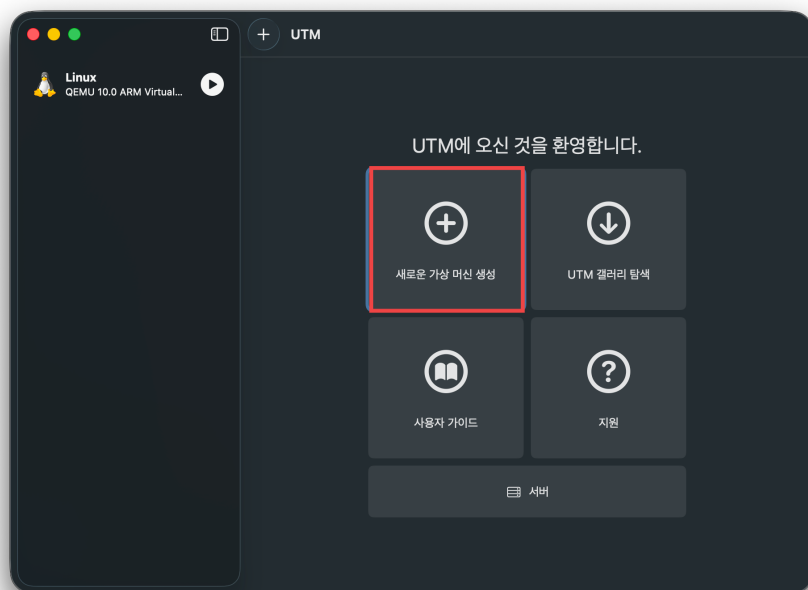
이 보고서의 화면과 절차는 Apple Silicon M4 기반 macOS와 UTM을 기준으로 한다. 설치가 끝난 뒤에는 가상머신에 SSH로 접속해 터미널 중심으로 실습한다.

1.2 UTM: Ubuntu 가상머신 생성

1.2.1 새로운 가상머신 생성

UTM 시작 화면에서 새로운 가상 머신 생성을 선택한다.

그림 1. UTM home screen for creating a new virtual machine (새 가상머신을 만드는 UTM 시작 화면)



하드웨어 가상화 기능을 사용하는 가상화 모드를 선택한다.

설치할 운영체제로 Linux를 선택한다.

가상머신 메모리를 8GB(8192MB)로 지정한다. 호스트의 다른 작업에 필요한 메모리를 고려해 가상머신에 전체 메모리를 할당하지 않는다.

부팅 이미지 유형에서 Boot from ISO image를 선택한다.

그림 2. UTM virtualization mode selection (UTM 가상화 모드 선택)

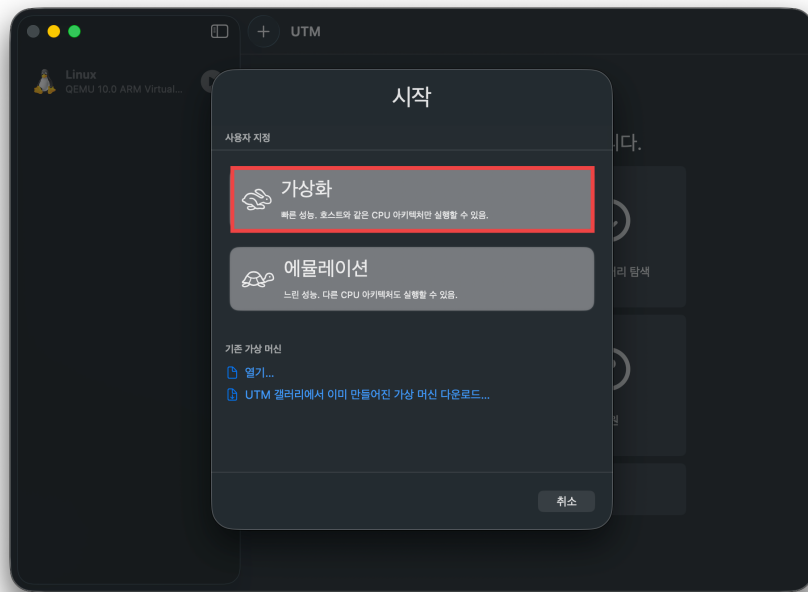


그림 3. Linux operating system selection in UTM (UTM Linux 운영체제 선택)

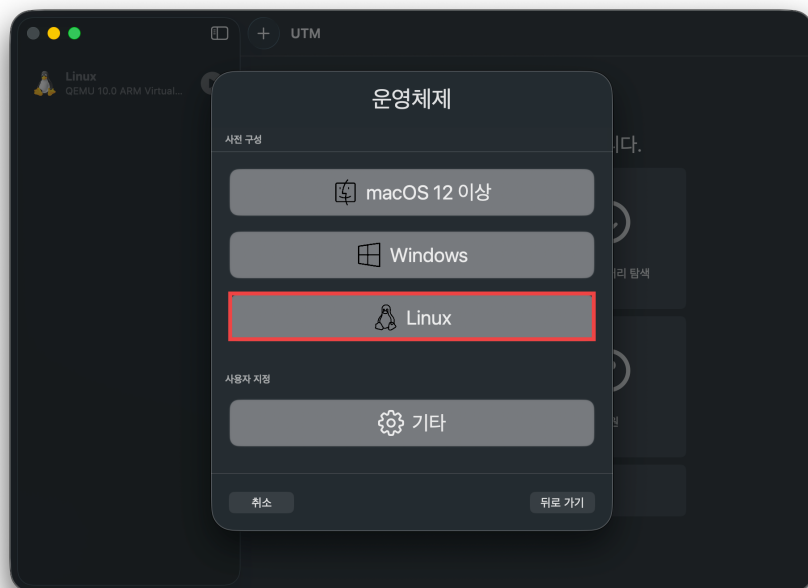


그림 4. Virtual machine memory set to 8 GB (가상머신 메모리 8GB 설정)

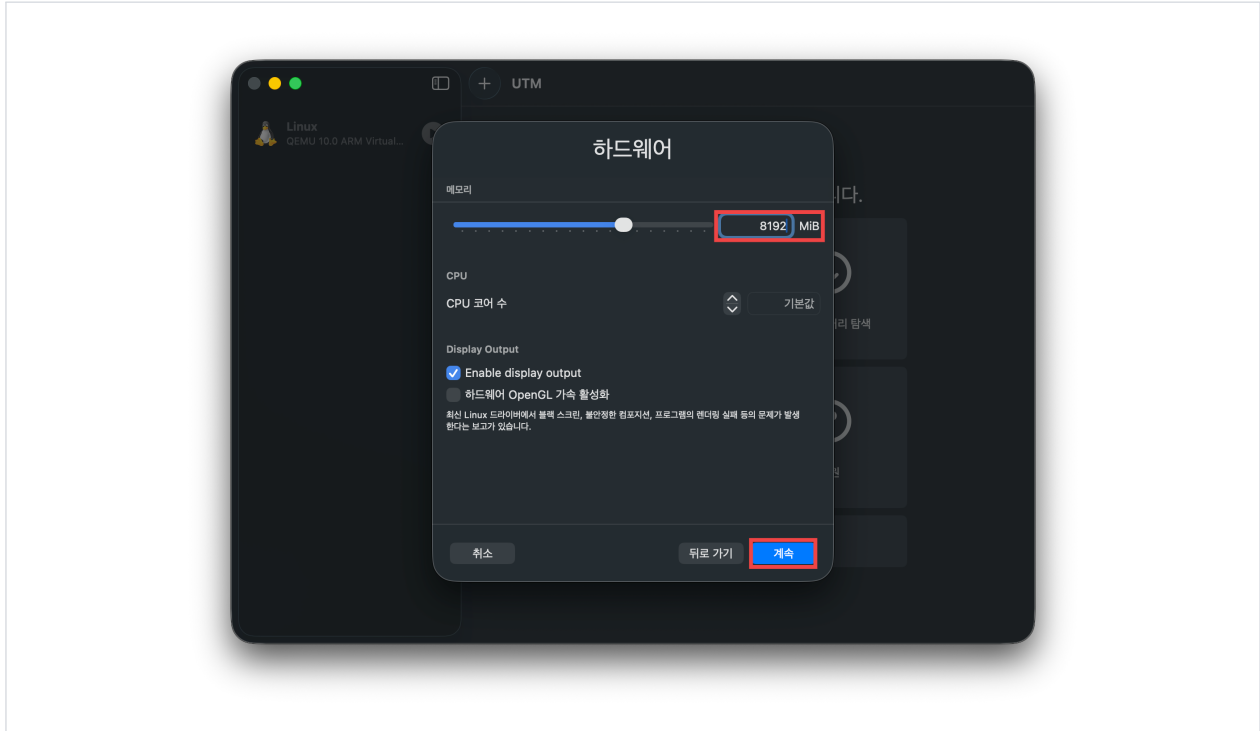
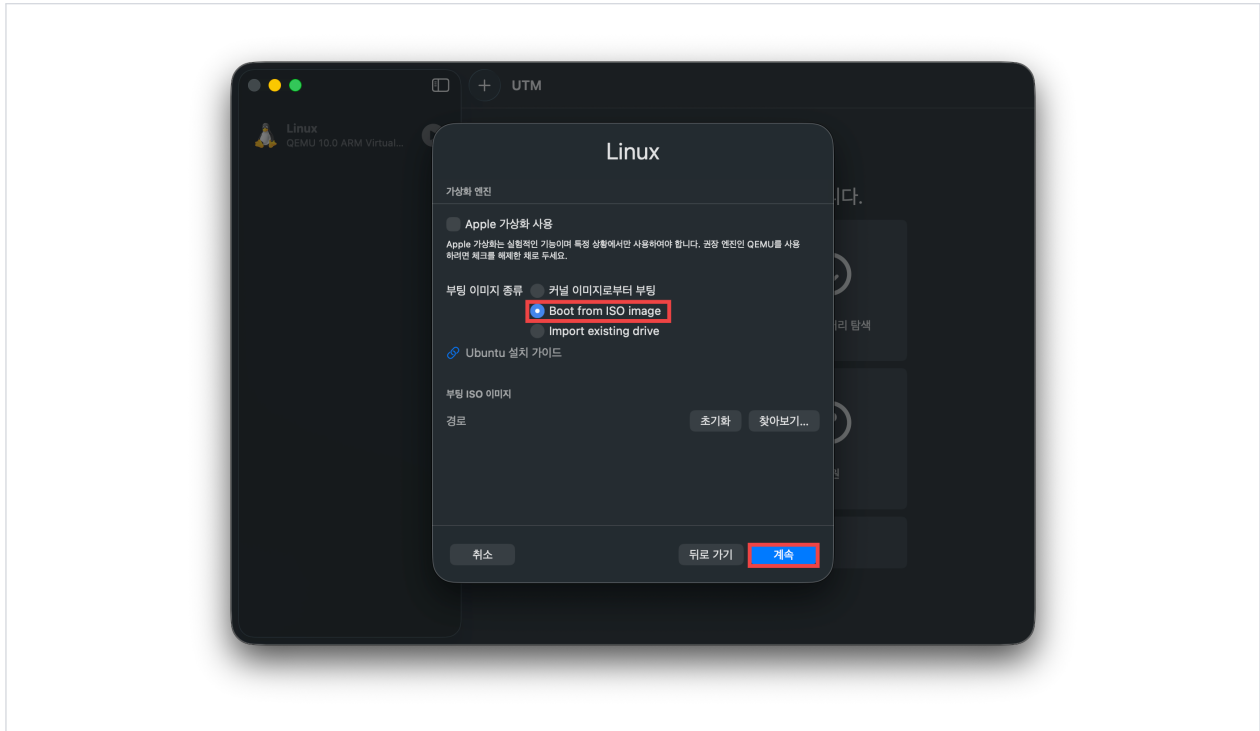


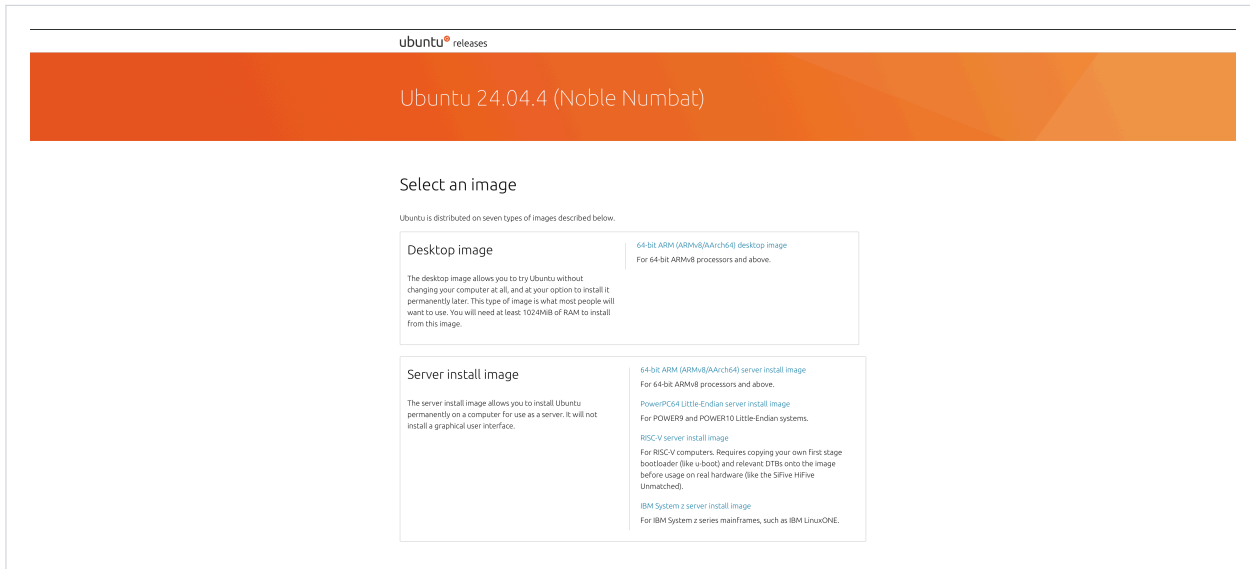
그림 5. Boot from ISO image option in UTM (UTM ISO 이미지 부팅 옵션)



1.2.2 ARM64 ISO 선택과 아키텍처 오류 방지

Apple Silicon M 시리즈는 ARM64(AArch64) 아키텍처를 사용하므로 Ubuntu Server의 64-bit ARM 서버 설치 이미지를 내려받아야 한다.

그림 6. Ubuntu 24.04.4 ARM64 server image download page (Ubuntu 24.04.4 ARM64 서버 이미지 다운로드 페이지)



! ISO 아키텍처 확인

Ubuntu Server 기본 다운로드 화면에서 받은 파일은 amd64 이미지일 수 있다. Apple Silicon용 가상머신에는 파일명에 arm64가 포함된 이미지를 사용해야 한다. amd64는 Intel·AMD의 x86-64용이며 ARM64 가상머신과 호환되지 않는다.

다음 화면은 기본 다운로드 페이지와 잘못된 선택한 amd64 ISO의 예이다.

UTM이 선택한 ISO의 amd64와 가상머신의 ARM64가 일치하지 않는다고 경고하면 ISO를 교체해야 한다. 호환되지 않는 ISO로 부팅하면 Ubuntu 설치 메뉴 대신 UEFI 셸이 나타날 수 있다.

이 경우 가상머신을 종료하고 UTM의 이동식 드라이브 메뉴에서 잘못된 ISO를 꺼낸 뒤 ARM64 ISO로 교체한다.

Boot from ISO image를 선택하고 파일명에 arm64가 포함된 Ubuntu Server ISO를 지정한다.

1.2.3 저장소와 가상머신 이름 설정

가상 디스크 크기는 64GB로 지정한다. 이 값은 가상머신에 제공되는 최대 디스크 용량이며, 호스트의 실제 사용량은 저장 방식과 가상머신 내부 데이터에 따라 달라질 수 있다.

공유 디렉터리는 호스트와 가상머신 사이에서 파일을 주고받을 때 사용한다. 이번 터미널 실습에 필수는 아니므로 필요할 때만 지정한다.

요약 화면에서 가상머신 이름을 Linux-Car-1로 변경하고, 운영체제와 CPU 아키텍처가 의도한 값인지 확인한 뒤 저장한다.

UTM 목록에서 생성된 가상머신을 선택하고 실행 버튼을 누른다.

그림 7. Default Ubuntu Server download page (Ubuntu Server 기본 다운로드 페이지)

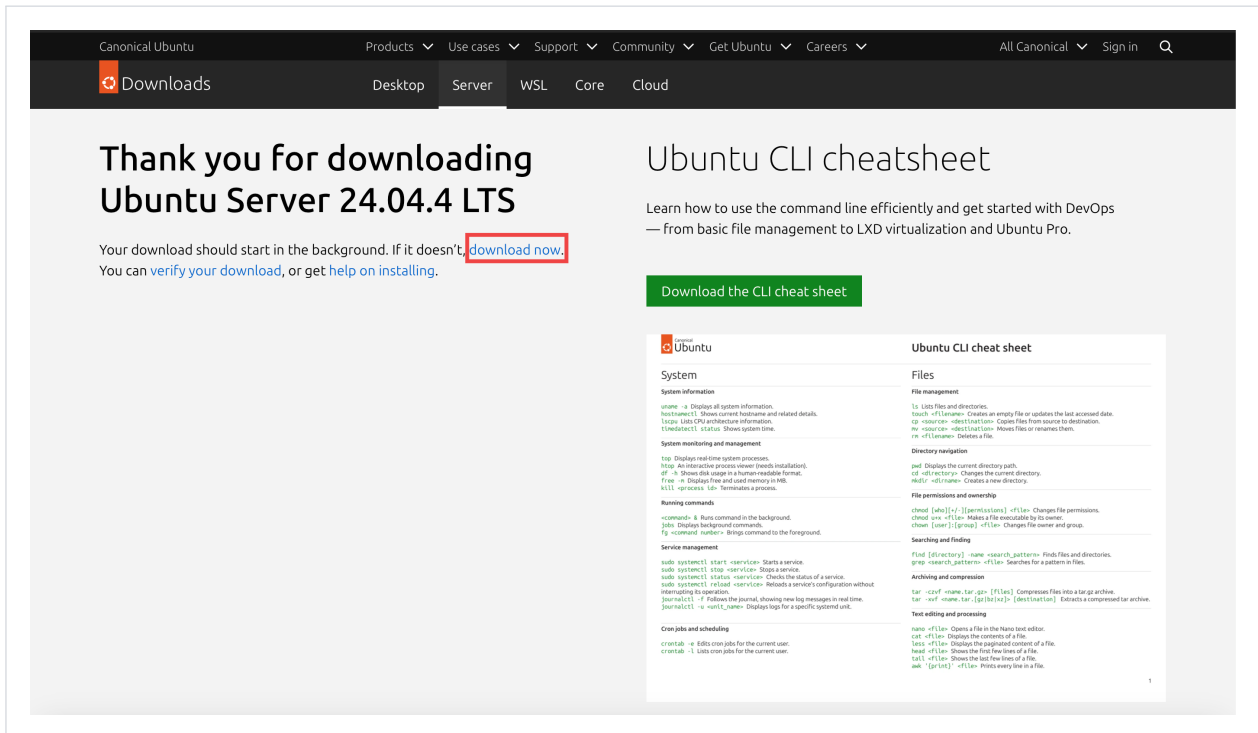


그림 8. AMD64 ISO selected on an ARM64 virtual machine (ARM64 가상머신에 잘못 선택한 AMD64 ISO)

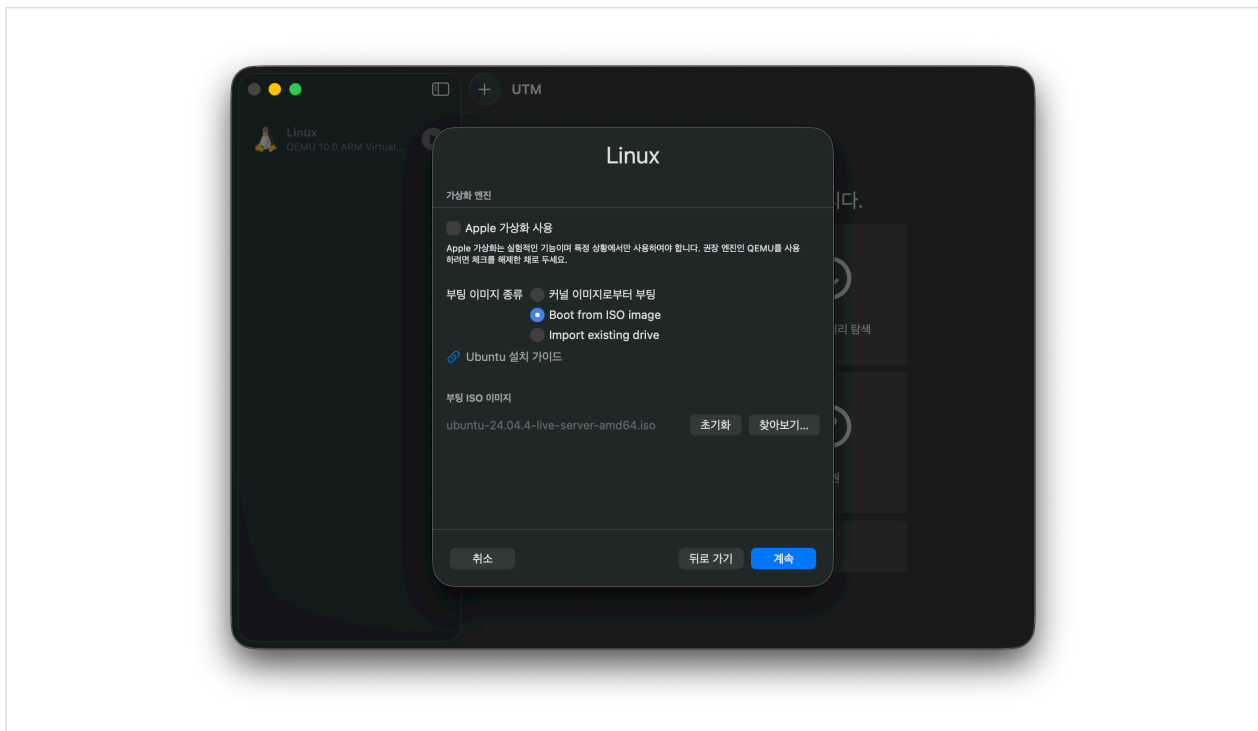


그림 9. UTM warning for an AMD64 and ARM64 architecture mismatch (AMD64와 ARM64 아키텍처 불일치 UTM 경고)

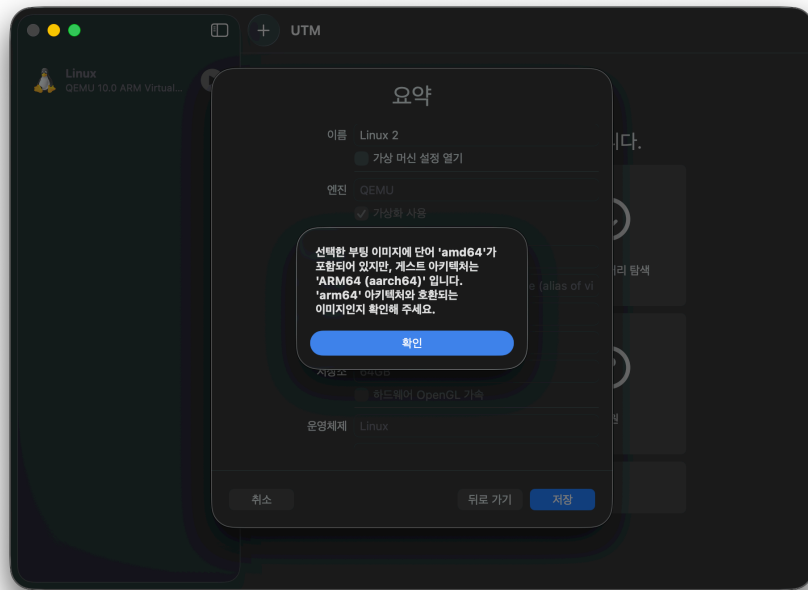


그림 10. UEFI shell displayed after an incompatible ISO boot (호환되지 않는 ISO 부팅 후 표시된 UEFI 셸)

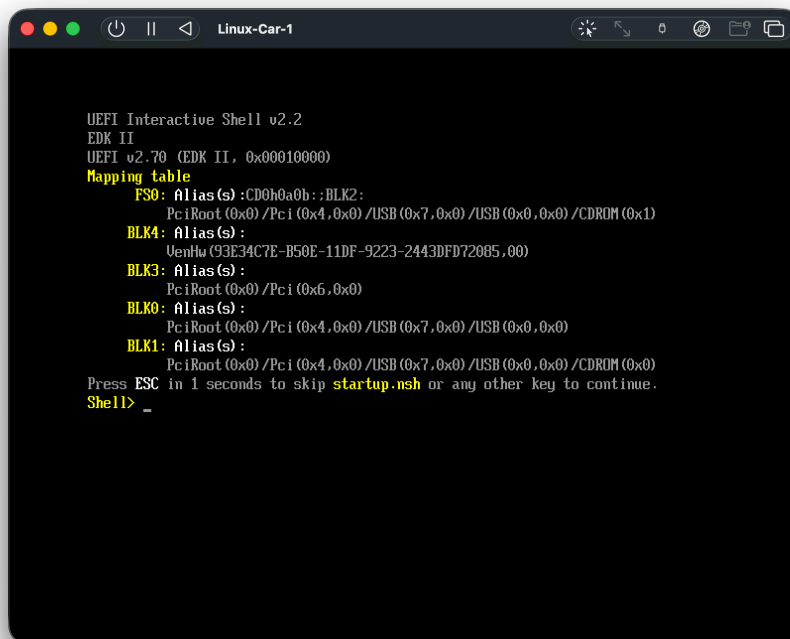


그림 11. UTM removable-drive menu for replacing the installation ISO (설치 ISO를 교체하는 UTM 이동식 드라이브 메뉴)

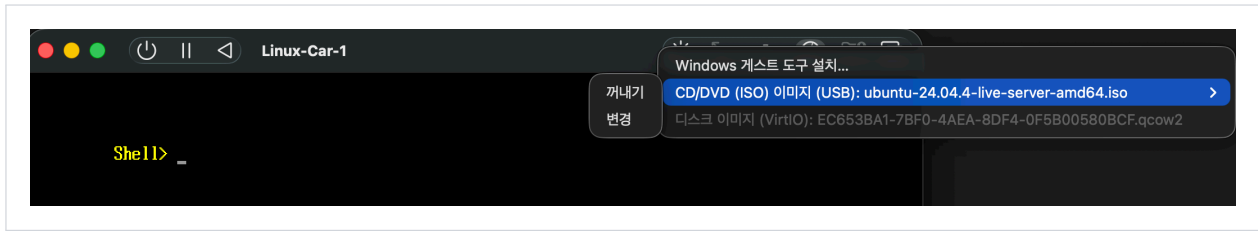


그림 12. ARM64 Ubuntu Server ISO selected in UTM (UTM에서 선택한 ARM64 Ubuntu Server ISO)

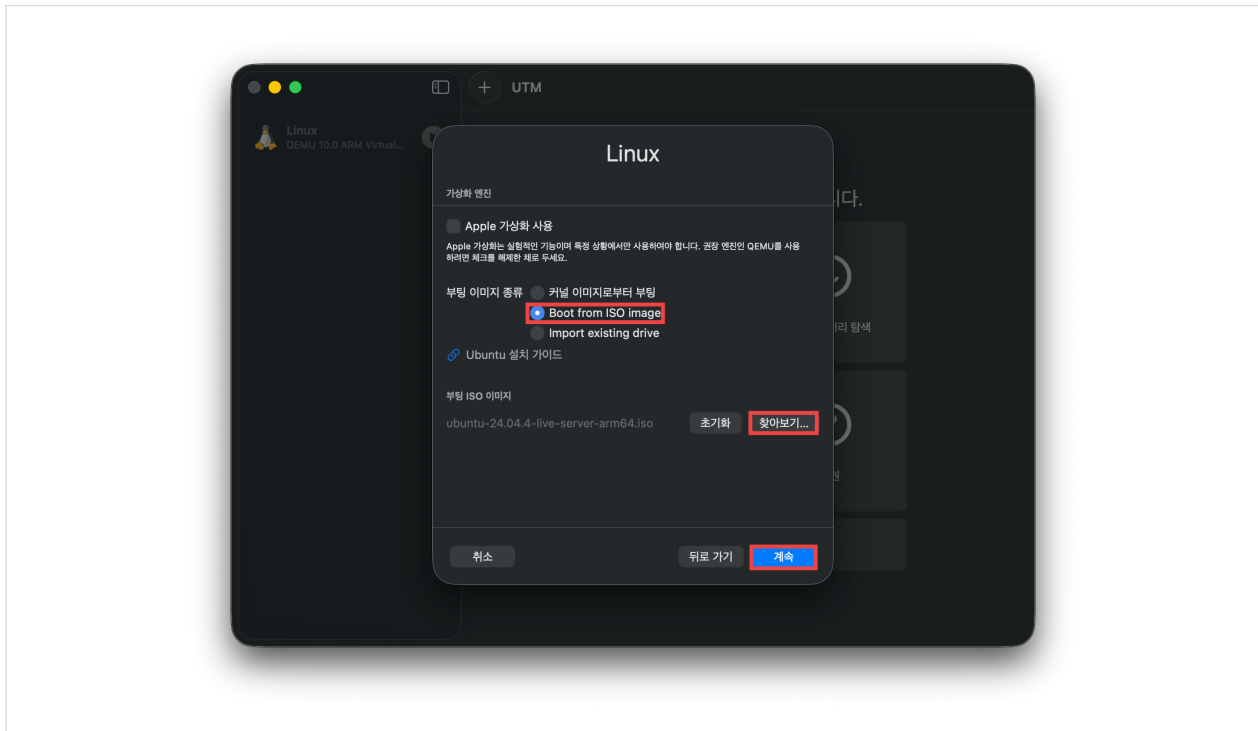


그림 13. Virtual machine storage set to 64 GB (가상머신 저장소 64GB 설정)

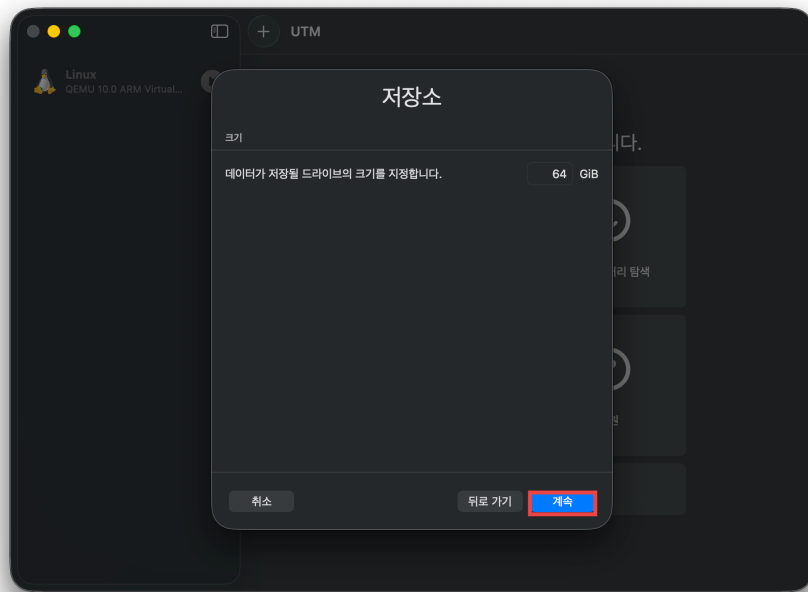


그림 14. Optional shared directory setting in UTM (UTM의 선택적 공유 디렉터리 설정)

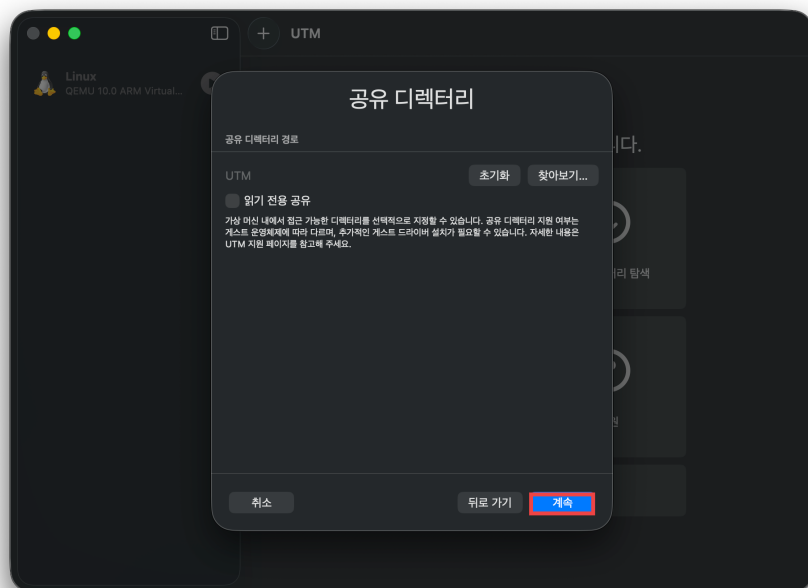


그림 15. Virtual machine summary and Linux-Car-1 name (Linux-Car-1 이름을 지정한 가상머신 요약)

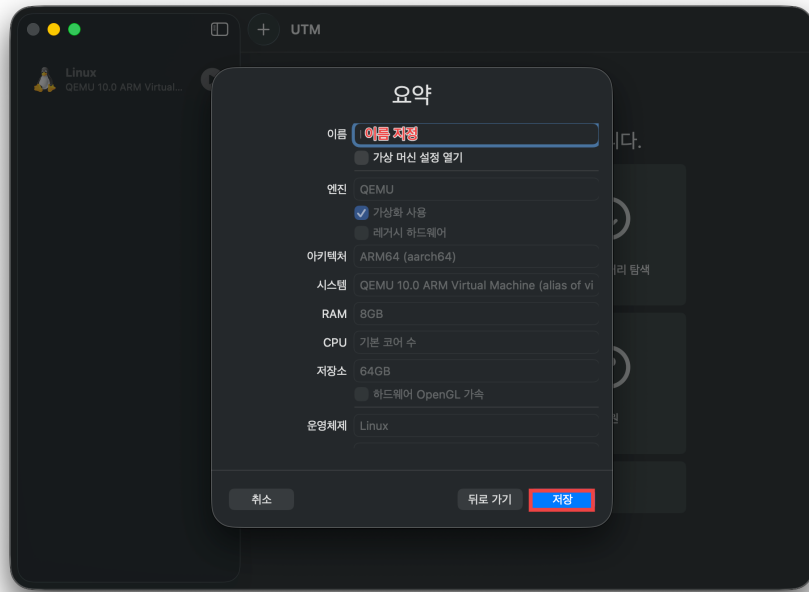
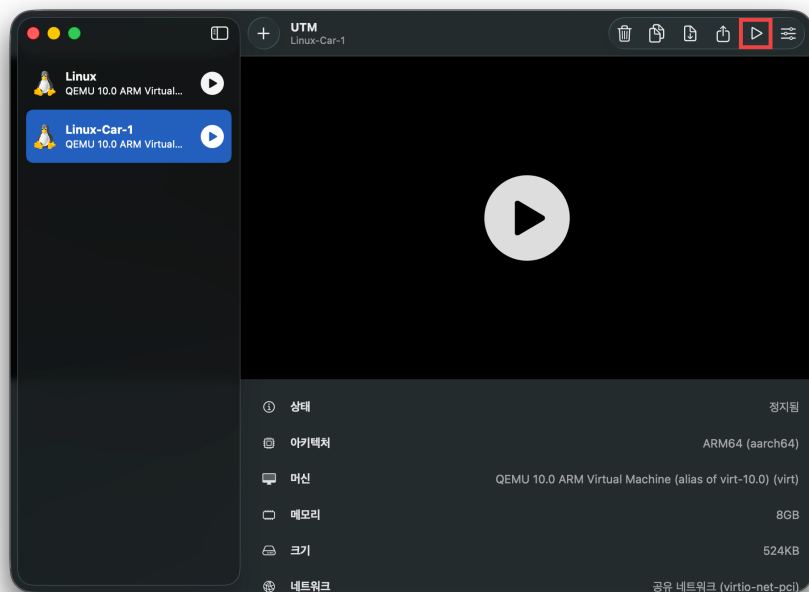


그림 16. Linux-Car-1 virtual machine ready to start (실행 준비가 완료된 Linux-Car-1 가상머신)

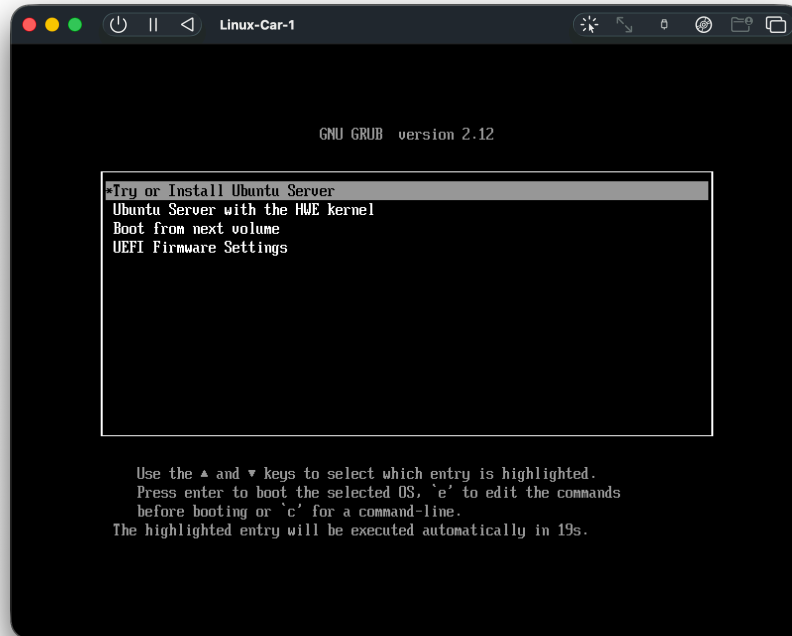


1.3 Ubuntu 설치

1.3.1 설치 프로그램 기본 설정

부팅 메뉴에서 Try or Install Ubuntu Server를 선택해 서버 설치를 시작한다.

그림 17. Ubuntu Server installer boot menu (Ubuntu Server 설치 프로그램 부팅 메뉴)



설치 언어는 English를 선택한다. 이 설정은 설치 프로그램의 표시 언어이며, 설치 후 시스템 로캘은 별도로 변경할 수 있다.

설치 프로그램 업데이트 화면에서는 현재 포함된 설치 프로그램으로 계속 진행한다. 운영체제 설치가 끝난 뒤 apt update와 apt upgrade를 실행해 패키지를 갱신한다.

키보드 레이아웃은 English로 설정한다. 터미널에서 특수문자를 입력할 때 실제 키 배열과 선택한 레이아웃이 일치하는지 확인한다.

설치 유형은 기본 Ubuntu Server를 선택한다.

1.3.2 네트워크와 미러 설정

네트워크 인터페이스가 DHCP로 IP 주소를 받은 것을 확인하고 기본 설정으로 진행한다. 이 IP 주소는 설치 후 SSH 접속에 사용하므로 기록해 둔다.

별도 프록시를 사용하지 않는 환경에서는 Proxy address를 비워 둔다.

기본 Ubuntu archive mirror의 주소와 연결 검사가 성공했는지 확인한 뒤 진행한다.

그림 20. English keyboard layout in the Ubuntu installer (Ubuntu 설치 프로그램의 영어 키보드 배열)

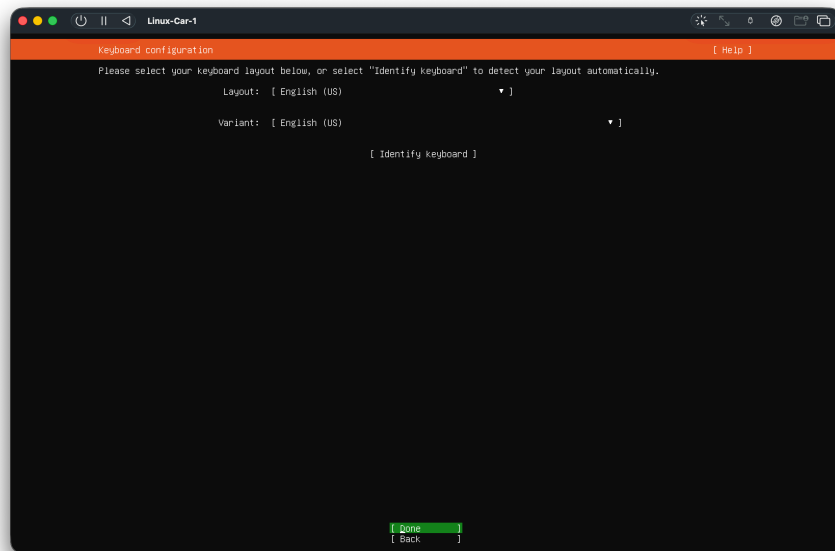


그림 21. Ubuntu Server installation type selection (Ubuntu Server 설치 유형 선택)

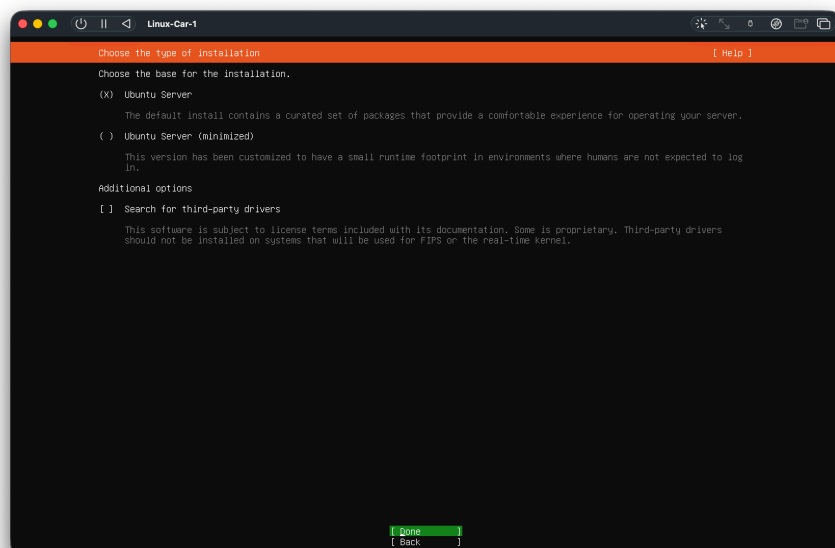


그림 22. DHCP network configuration in the Ubuntu installer (Ubuntu 설치 프로그램의 DHCP 네트워크 설정)

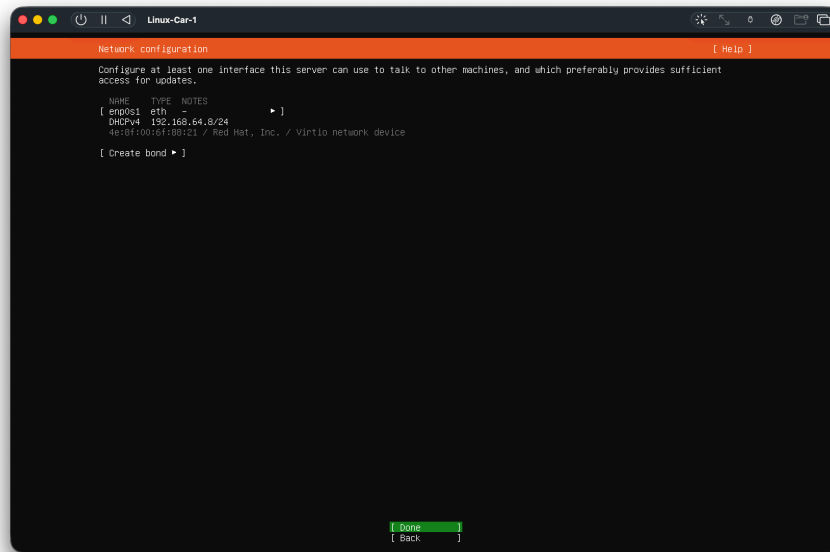


그림 23. Empty proxy configuration in the Ubuntu installer (Ubuntu 설치 프로그램의 빈 프록시 설정)

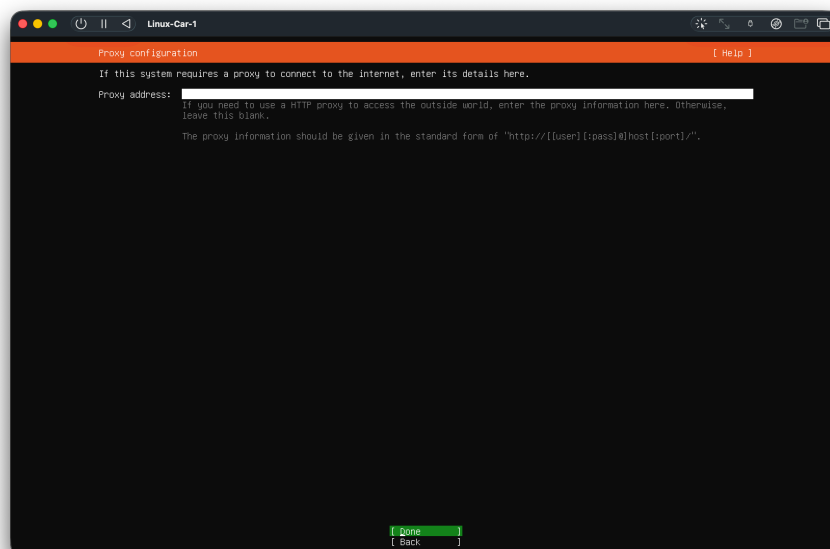
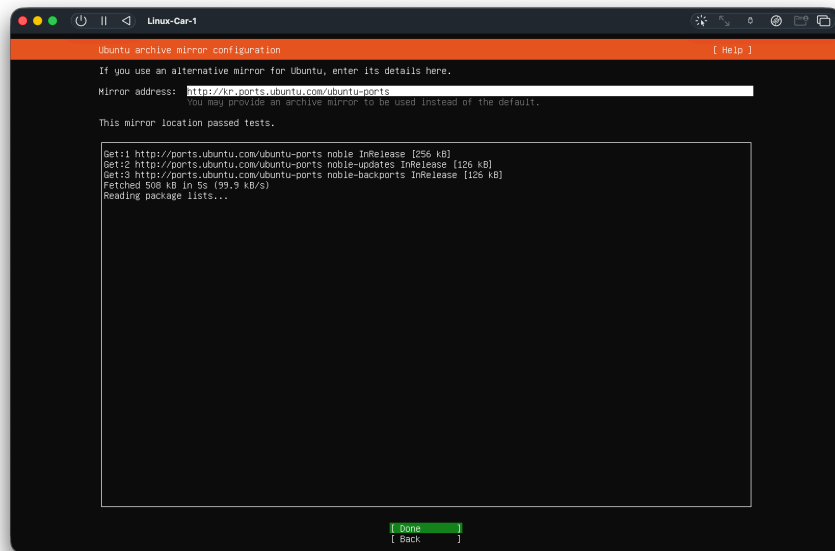


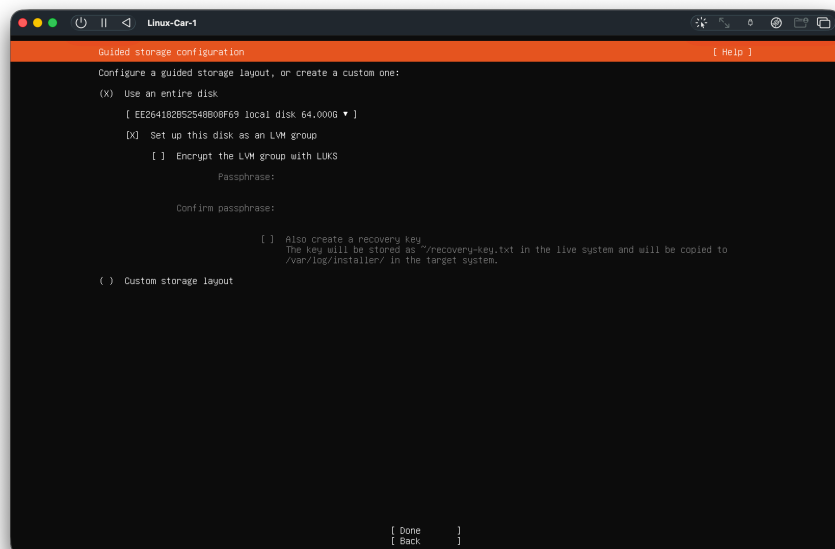
그림 24. Ubuntu archive mirror connectivity check (Ubuntu 패키지 미러 연결 확인)



1.3.3 저장소와 사용자 계정 설정

가상 디스크 전체를 사용하는 guided storage 구성을 선택한다. 여기서 포맷되는 대상은 UTM에 만든 가상 디스크이며, 선택한 디스크와 용량이 맞는지 확인해야 한다.

그림 25. Guided storage configuration for the virtual disk (가상 디스크의 자동 저장소 구성)



생성될 파티션과 마운트 지점을 검토한다. 루트 파일 시스템 /이 할당되어 있는지 확인하고 Done을 선택한다.

설치를 시작하면 선택한 가상 디스크의 기존 데이터가 삭제된다는 경고가 표시된다. 대상 디스크를 다시 확인하고 Continue를 선택한다.

그림 26. Final storage layout before installation (설치 전 최종 저장소 레이아웃)

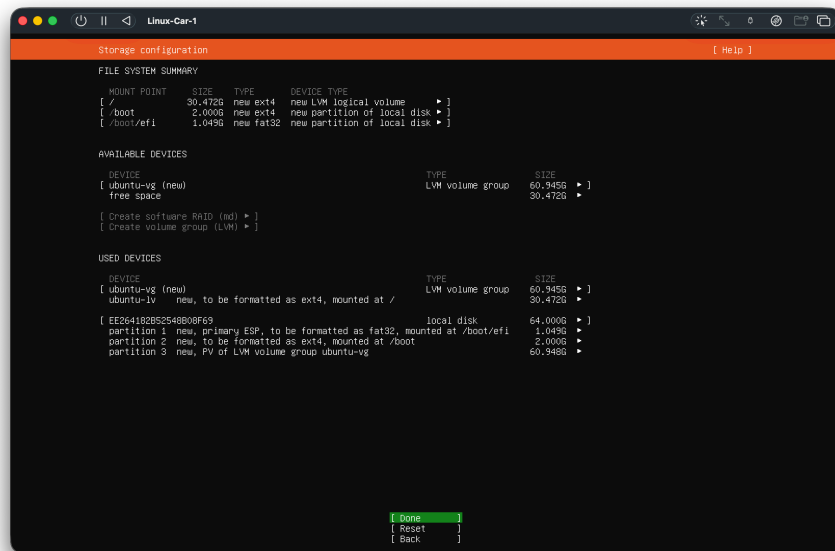
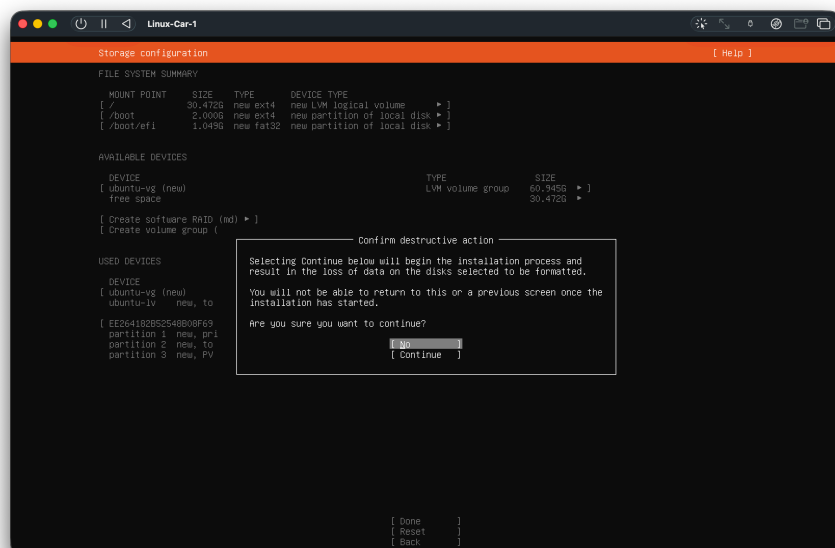
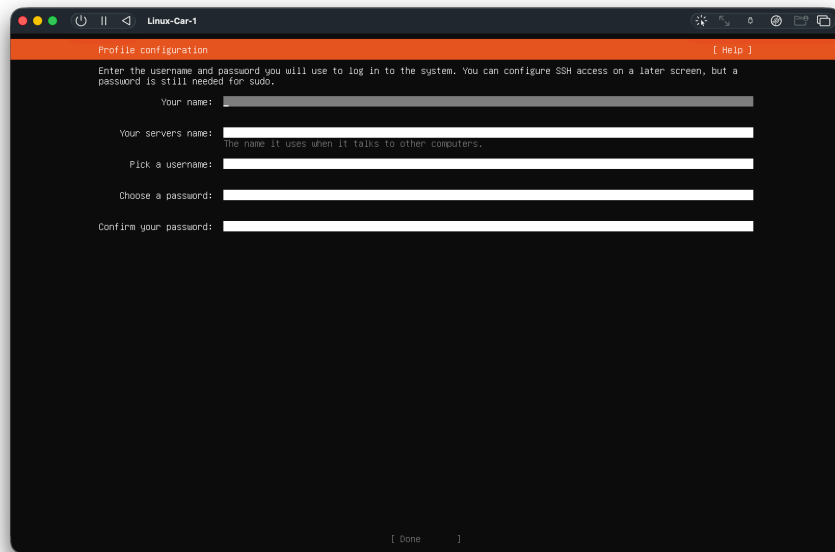


그림 27. Destructive storage action confirmation (저장소 변경 적용 확인)



사용자 이름, 서버 이름, 로그인 계정과 비밀번호를 입력한다. 이 실습에서는 SSH 예시에 맞춰 사용자 계정을 car1, 서버 이름을 car1-server로 사용한다.

그림 28. User profile and server name configuration (사용자 프로필과 서버 이름 설정)



Ubuntu Pro 연결은 이번 실습의 필수 항목이 아니므로 Skip for now를 선택한다.

원격 터미널 접속을 위해 Install OpenSSH server를 선택한다. SSH 키를 별도로 등록하지 않는 경우 설치 후 계정 비밀번호로 접속한다.

추가 server snap은 이번 실습에 필요하지 않으므로 선택하지 않고 진행한다.

1.3.4 설치 완료와 첫 로그인

패키지 설치와 시스템 설정이 완료될 때까지 기다린다.

Reboot Now가 활성화되면 재부팅을 진행한다.

재부팅 중 Failed unmounting cdrom.mount와 설치 미디어 제거 안내가 나타나면 UTM에 ISO가 연결된 상태라는 의미이다.

UTM의 이동식 드라이브 메뉴에서 Ubuntu 설치 ISO를 꺼내기한 뒤 가상머신 화면에서 Enter를 누른다. ISO를 분리하면 다음 부팅부터 가상 디스크의 Ubuntu가 실행된다.

재부팅이 완료되면 설치 중 만든 계정으로 로그인한다. 로그인 프롬프트와 시스템 정보가 표시되면 기본 설치가 완료된 것이다.

패키지 색인을 갱신하고 설치된 패키지를 업그레이드한다. net-tools는 ifconfig를 사용하기 위해 설치한다.

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y net-tools
ifconfig
```

그림 29. Skipping Ubuntu Pro during installation (설치 중 Ubuntu Pro 건너뛰기)

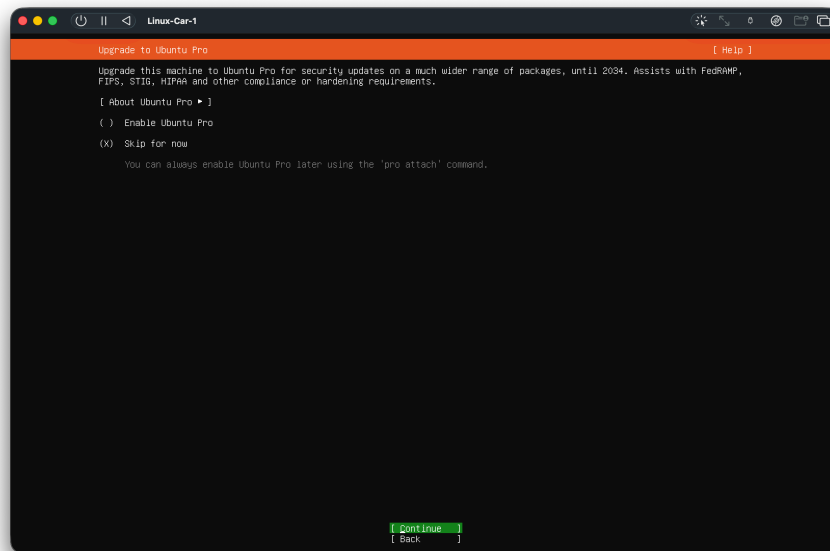


그림 30. OpenSSH server enabled in the Ubuntu installer (Ubuntu 설치 프로그램에서 OpenSSH 서버 활성화)

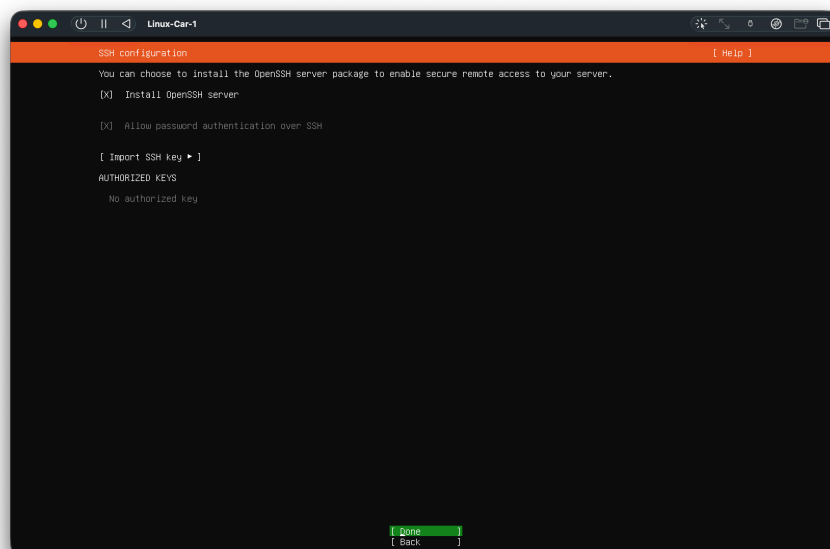


그림 31. Optional server snaps selection (선택적 서버 snap 목록)

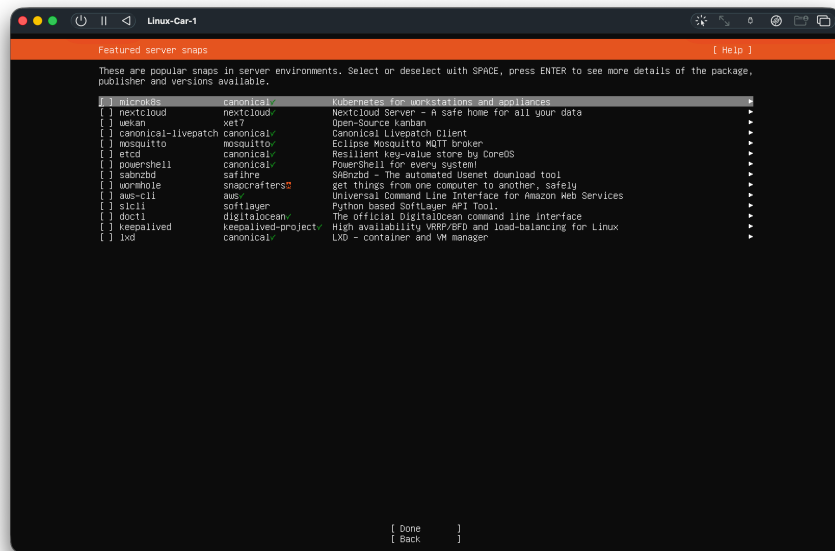


그림 32. Ubuntu Server installation progress (Ubuntu Server 설치 진행 화면)

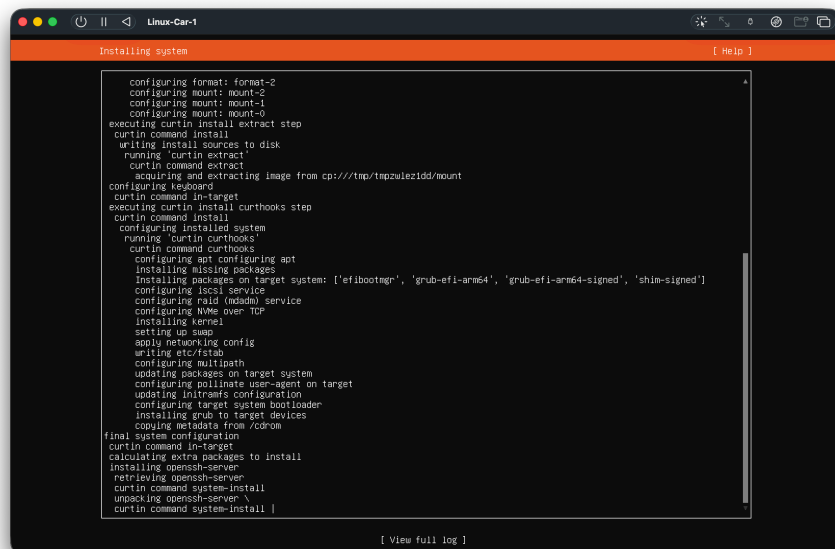


그림 33. Ubuntu Server installation completed (Ubuntu Server 설치 완료 화면)

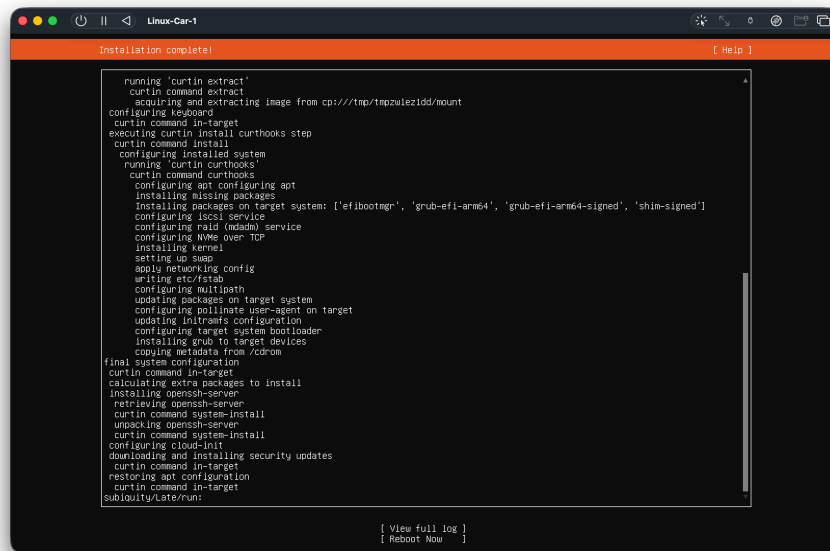


그림 34. Installation media unmount warning during reboot (재부팅 중 설치 미디어 마운트 해제 경고)

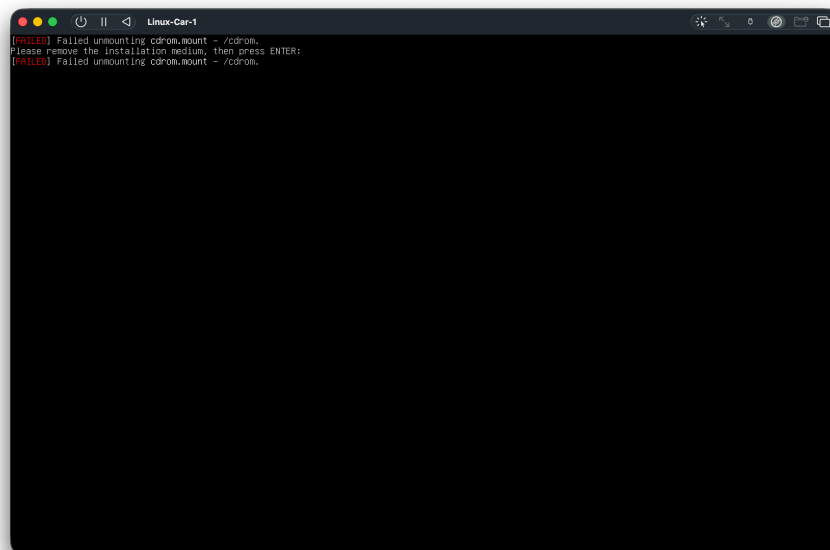


그림 35. Ejecting the Ubuntu installation ISO in UTM (UTM에서 Ubuntu 설치 ISO 꺼내기)

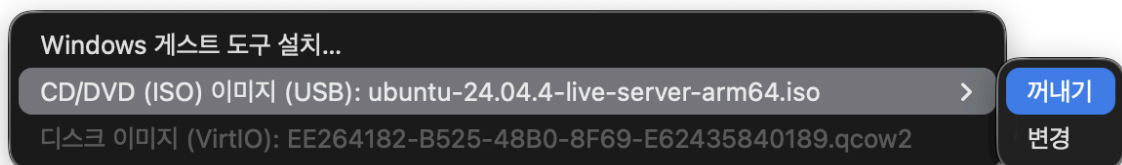
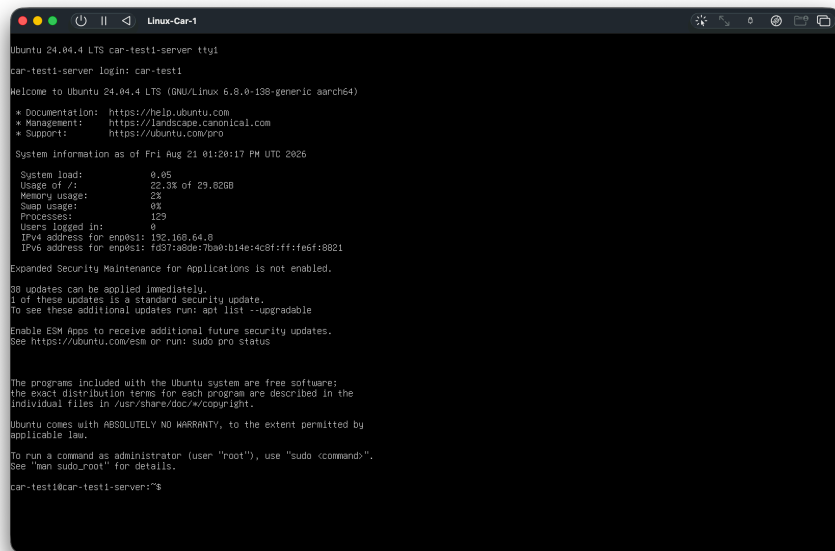


그림 36. First successful login to Ubuntu Server (Ubuntu Server 첫 로그인 성공 화면)



1.4 SSH 연결

호스트 PC의 터미널에서 가상머신의 DHCP 주소로 접속한다. <VM_IP>에는 설치 과정이나 ifconfig에서 확인한 실제 IP 주소를 입력한다.

```
ssh car1@<VM_IP>
```

다음과 같이 가상머신의 셸 프롬프트가 표시되면 SSH 연결이 정상이다.

```
car1@car1-server:~$
```

2 Docker 없이 vcan0과 can-utils 실행

vcan은 물리 CAN 장치 없이 CAN 프레임 송수신을 시험할 수 있는 Linux 커널 모듈이다. vcan0에서 전송한 프레임은 실제 차량 네트워크로 나가지 않고 현재 네트워크 네임스페이스 안에서만 처리된다.

2.1 vcan0 생성과 상태 확인

```

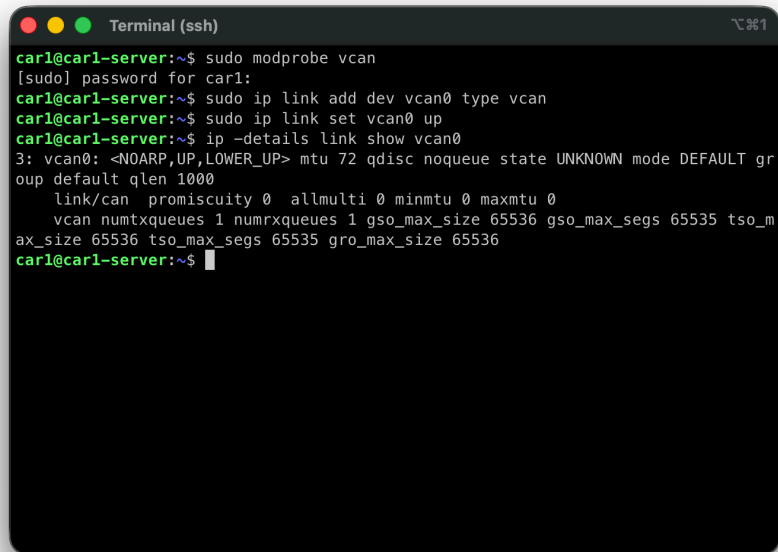
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set vcan0 up

ip -details link show vcan0

```

출력에 vcan0, UP, LOWER_UP, link/can이 표시되면 인터페이스가 생성되고 활성화된 것이다.

그림 37. Active vcan0 interface on the Ubuntu host (Ubuntu 호스트에서 활성화된 vcan0 인터페이스)



```

Terminal (ssh)
carl@carl-server:~$ sudo modprobe vcan
[sudo] password for carl:
carl@carl-server:~$ sudo ip link add dev vcan0 type vcan
carl@carl-server:~$ sudo ip link set vcan0 up
carl@carl-server:~$ ip -details link show vcan0
3: vcan0: <NOARP,UP,LOWER_UP> mtu 72 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/can promiscuity 0 allmulti 0 minmtu 0 maxmtu 0
        vcan numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 tso_max_size 65536 tso_max_segs 65535 gro_max_size 65536
carl@carl-server:~$

```

2.2 can-utils 설치

can-utils에는 CAN 프레임을 수신하는 candump와 전송하는 cansend가 포함된다.

```

sudo apt update
sudo apt install -y can-utils

```

2.3 Ubuntu 호스트에서 송수신 확인

Terminal 1에서 vcan0으로 들어오는 프레임을 대기한다.

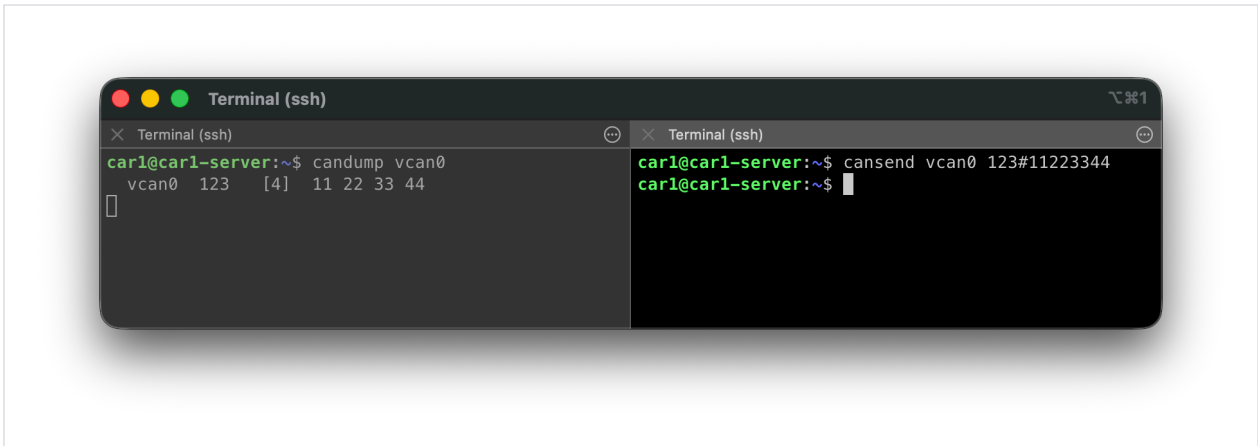
```
candump vcan0
```

Terminal 2에서 CAN ID 0x123, 데이터 11 22 33 44인 프레임을 전송한다.

```
cansend vcan0 123#11223344
```

Terminal 1에 vcan0 123 [4] 11 22 33 44가 표시되면 호스트 내부의 가상 CAN 송수신이 정상이다.

그림 38. Successful candump and cansend test on the Ubuntu host (Ubuntu 호스트의 candump와 cansend 송수신 성공)



3 Docker 설치

Docker Engine은 [Install Docker Engine on Ubuntu](#)의 apt 저장소 설치 절차를 사용한다.

3.1 Docker apt 저장소 설정

패키지 색인을 갱신하고 저장소 설정에 필요한 패키지를 설치한다.

```
sudo apt update
sudo apt install ca-certificates curl
```

Docker 공식 GPG 키를 등록한다.

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

현재 Ubuntu 코드명과 CPU 아키텍처를 사용해 Docker 저장소를 추가한다.

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

새 저장소의 패키지 정보를 반영한다.

```
sudo apt update
```

3.2 Docker Engine과 Compose 설치

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Docker 서비스가 실행 중인지 확인한다.

```
sudo systemctl status docker
```

3.3 Docker 설치 확인과 사용자 권한 설정

Docker 클라이언트와 데몬 연결 상태를 확인한다.

```
docker --version
docker info
```

실행 당시 클라이언트 버전은 다음과 같이 확인되었다.

```
Docker version 29.7.2, build a7dcaa6
```

docker info에서 다음 오류가 표시되면 Docker가 설치되지 않은 것이 아니라 현재 사용자에게 /var/run/docker.sock 접근 권한이 없는 상태이다.

```
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
```

현재 사용자를 docker 그룹에 추가하고 새 로그인 셸에 그룹 변경을 적용한다.

```
sudo usermod -aG docker "$USER"
newgrp docker
```

docker 그룹은 Docker 데몬을 제어할 수 있는 높은 권한을 제공하므로 신뢰할 수 있는 사용자만 추가한다. 그룹 목록과 데몬 연결을 다시 확인한다.

```
groups
docker info
docker run --rm hello-world
```

그룹 목록에 docker가 포함되고 docker info에 Client와 Server 정보가 함께 표시되며 hello-world가 성공 메시지를 출력하면 설치와 권한 설정이 완료된 것이다.

3.4 Ubuntu 24.04 이미지와 컨테이너 실행

Ubuntu 24.04 이미지를 내려받고 목록에서 확인한다.

```
docker pull ubuntu:24.04
docker images
```

실습에서 확인한 이미지 정보는 다음과 같다.

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
ubuntu:24.04	33ceb71981b6	141MB	30.8MB	

ubuntu1 컨테이너를 백그라운드로 실행한다. `--network host`는 컨테이너가 Ubuntu 가상머신의 네트워크 네임스페이스를 공유하도록 하며, 이후 호스트의 `vcan0`을 컨테이너에서 사용하기 위해 필요하다.

```
docker run -d \
  --name ubuntu1 \
  --network host \
  ubuntu:24.04 \
  sleep infinity
```

컨테이너 상태를 확인한다.

```
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
825461431a98	ubuntu:24.04	"sleep infinity"	8 seconds ago	Up 7 seconds		ubuntu1

자주 사용하는 컨테이너 생명주기 명령은 다음과 같다.

```
docker stop ubuntu1
docker start ubuntu1
docker restart ubuntu1
docker logs ubuntu1
```

컨테이너를 삭제할 때는 먼저 정지한 뒤 `docker rm ubuntu1`을 실행한다. 실행 중인 컨테이너 셀에 접속하려면 다음 명령을 사용한다.

```
docker exec -it ubuntu1 bash
```

4 Docker에서 vcan0 확인

Docker의 기본 bridge 네트워크는 호스트와 분리된 네트워크 네임스페이스를 사용한다. 반면 `--network host`로 실행한 `ubuntu1` 컨테이너는 Ubuntu 가상머신의 네트워크 인터페이스를 공유하므로, 호스트에서 먼저 만든 `vcan0`도 확인할 수 있다.

컨테이너에 네트워크와 CAN 도구를 설치한다.

```
docker exec -it ubuntu1 bash
apt update
apt install -y iproute2 can-utils
ip link show vcan0
exit
```

다음 오류가 나오면 컨테이너 생성만 완료되었을 뿐 Ubuntu 호스트에 `vcan0`이 아직 없거나, 컨테이너가 `host network`로 실행되지 않은 상태이다.

```
Device "vcan0" does not exist.
```

Ubuntu 호스트에서 `vcan0`의 존재와 상태를 확인한다. 인터페이스가 없다면 앞 절의 생성 명령을 먼저 실행한다.

```
ip -details link show vcan0
```

컨테이너의 네트워크 모드가 `host`인지 확인한다.

```
docker inspect -f '{{.HostConfig.NetworkMode}}' ubuntu1
```

```
host
```

두 조건을 만족한 뒤 컨테이너에서 다시 확인하면 `vcan0`이 표시된다.

```
docker exec -it ubuntu1 ip link show vcan0
```

```
4: vcan0: <NOARP,UP,LOWER_UP> mtu 72 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen
1000
    link/can
```

4.1 Docker 안에서 cansend와 candump 테스트

Terminal 1에서 컨테이너의 `candump`를 실행한다.

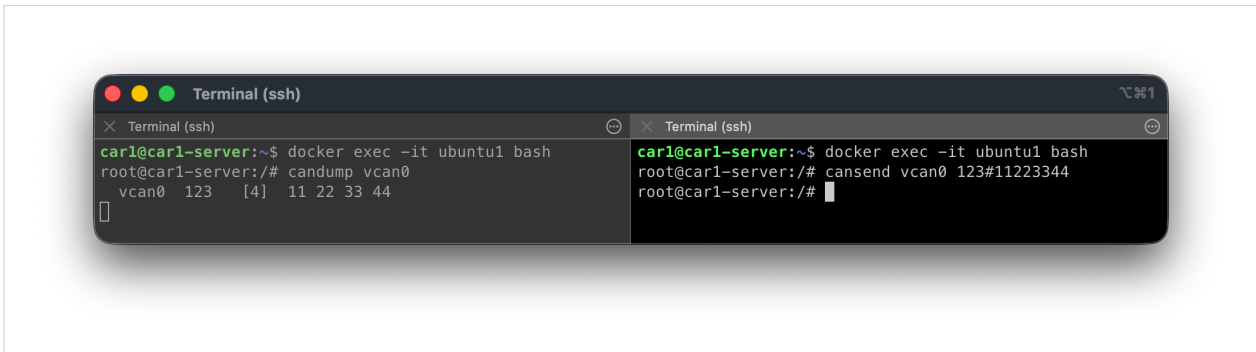
```
docker exec -it ubuntu1 bash
candump vcan0
```

Terminal 2에서 같은 컨테이너의 `cansend`를 실행한다.

```
docker exec -it ubuntu1 bash
cansend vcan0 123#11223344
```

Terminal 1에 CAN ID 123과 데이터 11 22 33 44가 표시되면 Docker 컨테이너가 Ubuntu 호스트의 `vcan0`을 공유하며 CAN 프레임을 정상적으로 송수신한 것이다.

그림 39. Successful candump and cansend test inside Docker (Docker 컨테이너 내부의 candump와 cansend 송수신 성공)



5 실습 결과

1. Apple Silicon에 맞는 ARM64 Ubuntu Server ISO로 UTM 가상머신을 설치하고 SSH 접속을 확인했다.
2. Ubuntu 호스트에 `vcan0`을 생성하고 `candump`와 `cansend`로 CAN 프레임의 로컬 송수신을 확인했다.
3. `--network host`로 실행한 Docker 컨테이너에서 호스트의 `vcan0`을 확인하고 동일한 CAN 프레임 송수신 시험을 완료했다.